

System Specification: Transformative Engine for Smart Testing

Version history

Version	Date	Author	Description
1.0	25.11.2025	Mikael Alamäki	First draft
1.1	30.11.2025	Mikael Alamäki	Added also screenshot from C4 architechture models just in case the link doesnt work

Approved by:

Nicolas Kivelä

Table of Contents

System Specification: Transformative Engine for Smart Testing.....	1
Table of Contents.....	1
1. High-Level Layout	2
2. Component Specifications	3
3. System Architecture (C4 Model).....	4
4. API Specification (Endpoints).....	6
5. Appendix.....	7

1. High-Level Layout

The application is a Single Page Application (SPA) designed to automate the creation of Robot Framework test scripts. It is divided into three primary functional zones:

1. Configuration Header: Ingestion point for requirements files and target system url.
2. Left Panel (Scenario Engine): Interface for managing generated BDD scenarios and initiating locator fetching based on the scenarios.
3. Right Panel (Code Engine): Output terminal for the generated Robot Framework code.

2. Component Specifications

Zone A: Configuration Header (Top Bar) Reference: Header Configuration State

Goal: Ingest the raw requirements file and target URL parameters.

Input Fields:

Field name	Type	Validation/Behaviour
Add requirements file	File Upload	Accepts .pdf or .txt formats. Required input
URL	Text input	Must be a valid url. Required input

Security Constraint & Design Decision:

- Credential Handling: The Username and Password fields are intentionally disabled/commented out in the current version. This is a strict architectural decision to prevent the leakage of sensitive credentials to third-party AI providers (LLMs) which are used to generate navigation context.

Primary Action: Button: Process data (Initially disabled until file and URL are present).

Behavior: Calls POST /requirements to upload and process the file.

Secondary Action: Button: Reset Inputs. Behavior: Clears all header fields.

Zone B: Feature Selection Modal (Overlay) Reference: Feature Selection Dialog

Trigger: Appears immediately after the Process data action completes. Data Source: Populated by GET /requirements which fetches the extracted "Requirement Topics".

UI Elements:

- Title: "Requirements have been processed"
- Instruction: "Choose the feature you want to continue making BDD scenarios and tests for."
- List: Radio button list of features.

Actions:

- Cancel: Closes modal.
- Continue: Calls POST /bdd_scenarios/generate/{id} (where ID is the requirement ID) to generate initial scenarios for the selected feature.

Zone C: BDD Scenarios Panel (Left Column) Reference: Scenario Management Interface

State: Populated by GET /bdd_scenarios. Display: Scrollable list of "Scenario Cards" with Gherkin syntax (Given/When/Then).

Card Actions:

- Edit: Calls PUT /bdd_scenarios to update text.
- Delete: Calls DELETE /bdd_scenarios to remove a card.

Panel Actions (Top Right):

1. Button: Fetch Locators Behavior: Calls POST /scraper/start to trigger the headless browser. The frontend may poll GET /scraper/fetch_locators to update the UI. Visual Feedback: Loading spinner required.
2. Button: Generate Tests Behavior: Calls POST /test_scripts/generate to compile the final script.

Zone D: Generated Code Panel (Right Column) Reference: Code Output Terminal

Active State: Displays syntax-highlighted Robot Framework code. Data Source: Populated by the response from POST /test_scripts/generate or GET /test_scripts.

Actions:

- Copy: Copies code to clipboard.
- Reset Session: Resets state.

3. System Architecture (C4 Model)

The system follows a containerized microservices architecture using the C4 model for documentation.

Level 1: System Context Goal: Define the system boundary and external interactions.

- User: QA Engineer/Developer providing requirements and URLs.
- Smart Tester (System): The application core.
- External Systems:
 - Web Page Under Test: The target website that the system scrapes for locators.
 - AI Provider API: Configurable Generative AI service (e.g., Gemini, OpenAI, etc.) used for NLP tasks (Scenario generation, Code generation) and intelligent locator analysis.

Level 2: Container Architecture Goal: Define the high-level technology choices.

- Frontend Container: Tech: Vue.js Responsibility: Renders the SPA, handles user input, communicates with Backend via REST API.
- Backend Container: Tech: Python (FastAPI) Responsibility: Orchestrates AI calls, manages browser scraping, stores session state.
- Deployment: Both containers are orchestrated locally using Docker Compose.

Level 3: Backend Component Architecture Goal: Define the internal modular logic of the Python Backend. All components leverage a configurable AI Provider to enhance accuracy and context awareness.

Component: Requirement Input Handling Responsibility: Parses PDF/TXT files. Uses AI to chunk text and extract relevant features/metadata intelligently. Dependencies: Receives File Input, Calls AI Provider API.

Component: BDD Generator Responsibility: Uses AI to convert raw text chunks into Gherkin-style BDD scenarios. Dependencies: Calls AI Provider API.

Component: Locator Retrieval Responsibility: Runs a headless browser instance. Uses AI to analyze the DOM and intelligently match elements to natural language steps. Dependencies: Visits Target Website, Calls AI Provider API.

Component: Test Script Generator Responsibility: Merges BDD Scenarios and Locators. Uses AI to produce valid Robot Framework syntax. Dependencies: Calls AI Provider API.

Component	Responsibility	Dependencies
Requirement Input Handling	Parses PDF/TXT files. Detects features. Creates summaries for them. Extracts requirements for each feature	Receives File Input, Calls AI Provider API using RequirementAgent
BDD Generator	Uses AI to convert requirements into Gherkin-style BDD scenarios	BDDGeneratorAgent that uses AI providers API.
Locator Retrieval	Runs a headless browser instance. Creates a navigation plan to fulfill the scenarios steps. Navigates and scrapes locators needed in the test scripts.	TaskAgent, NavigationAgent, LocatorAgent. All uses the AI providers API.
Test Script Generator	Merges BDD Scenarios and Locators. Uses AI to produce valid Robot Framework syntax	ScriptGenAgent that uses the AI providers API

4. API Specification (Endpoints)

The frontend communicates with the backend via the following REST endpoints.

Requirements Module

- POST /requirements Payload: File (PDF/TXT), Metadata. Function: Process Requirements.
- GET /requirements Function: Fetch Requirement Topics (Features).

BDD Scenarios Module

- GET /bdd_scenarios Function: Read BDD Scenarios by Feature.
- POST /bdd_scenarios Payload: Scenario Object. Function: Create manual BDD Scenario.
- PUT /bdd_scenarios Payload: Updated Scenario Object. Function: Update BDD Scenario text.
- DELETE /bdd_scenarios Payload: Scenario ID. Function: Delete BDD Scenario.

- POST /bdd_scenarios/generate/{id} Function: Trigger AI generation of scenarios for a specific Requirement ID.

Locators (Scraper) Module

- POST /scraper/start Function: Initialize the scraper process on the target URL.
- GET /scraper/fetch_locators Function: Get all found locators (XPath/CSS).
- GET /scraper/fetch_all_locator_data Function: Get detailed element data.

Test Scripts Module

- POST /test_scripts/generate Function: Compile final Robot Framework test.
- GET /test_scripts Function: Fetch previously generated tests.

5. Appendix

Repository Resources & Documentation The project repository's README.md serves as the primary operational guide. It contains detailed instructions on:

- Usage & Running: Steps to spin up the application using Docker Compose.
- Database: Configuration and management of the underlying storage.
- Environment Configuration: Setting up environment variables (including AI Provider keys).
- Debugging: Accessing screenshots and video recordings generated during the locator retrieval process for troubleshooting.

System Links

- Repository: <https://github.com/NicolasKivela/smart-tester>
- Frontend Application: Primary: <http://localhost:5173> Secondary: <http://localhost:5174>
- Backend API Documentation (Swagger/OpenAPI): Primary: <http://localhost:8000/docs> Alternate: <http://localhost:8010/docs>

C4 Model Documentation For a deeper dive into the architectural decisions and component interactions, please refer to the detailed C4 Model diagrams.

- C4 Model Reference: [C4-Model-IcePanel-Link](#)

C4 Model screenshots from different levels (from 1 to 3):

